

Planning and Reinforcement Learning in AI Systems

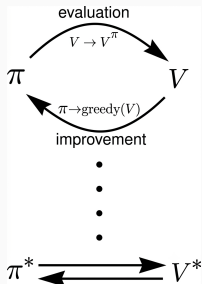
(PARLAI)

Lesson 7a - Model-Free RL (part 1)

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Reminder: The Unified Structure of an RL Algorithm



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Bias and Variance: Two Sources of Estimation Error

In reinforcement learning, we estimate quantities such as

$$V\pi(s), \quad Q^\pi(s, a).$$

Since these estimates are based on finite data, they are subject to two fundamental sources of error.

Bias

Systematic error. How close is the estimator's expected value to the true value?

Variance

Random error. How much would the estimate change if we collected a different set of experiences?

Learning algorithms strive for both low bias and low variance, but reducing one often increases the other.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Formal Definitions

Suppose $\hat{\theta}$ estimates an unknown quantity θ .

Bias

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

Measures the systematic error of the estimator.

Variance

$$\text{Var}(\hat{\theta}) = \mathbb{E} \left[\left(\hat{\theta} - \mathbb{E}[\hat{\theta}] \right)^2 \right].$$

Measures how much the estimate changes across different datasets.

Bias measures accuracy. Variance measures consistency.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Model Free RL: The Setup

The Setup for Model-Free RL

We want to learn a policy that achieves some objective, but can't/don't want to learn the environment dynamics.

We may be able to run many trajectories in the environment or only have access to a simulator that we can run to collect data (we need a lot of it!).

We can adopt several approaches in this setup. We start with **Monte-Carlo**.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo Simulation-Rollout

Each rollout $\tau = s_0, a_1, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T$ involves simulating a sequence of actions, typically by selecting actions according to some policy (e.g., uniformly at random, or based on a heuristic) until a terminal state is reached.

Monte Carlo



Model Free RL:
The Setup

Monte Carlo
Value Estimation

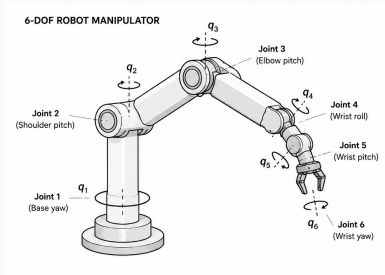
Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example



Model Free RL:
The Setup

Monte Carlo
Value Estimation

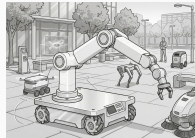
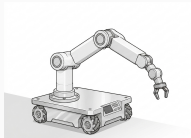
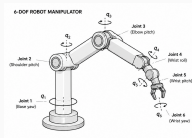
Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo Simulation-Rollout

- Rollouts are used to statistically analyze the behavior of complex systems and processes that are influenced by random variables.
- Typically, a large number of simulations are used to estimate the probabilities and distributions of different outcomes.
- Used in different algorithms (such as MCTS)

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo Simulation-Rollout

How does value propagate for a rollout

$$\tau = s_0, a_1, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo Simulation-Rollout

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

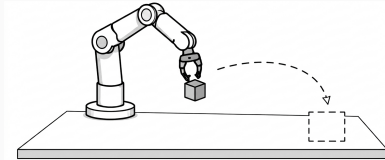
How does value propagate for a rollout

$$\tau = s_0, a_1, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T$$

The return for each state is the total [discounted] future reward from each time step: $G_t = r_{t+1} + \gamma G_{t+1}$ with $G_T = 0$.

When is this computed?

Example



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

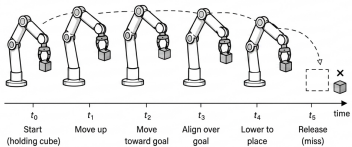
Monte Carlo
Control

Guarantees

Conclusion

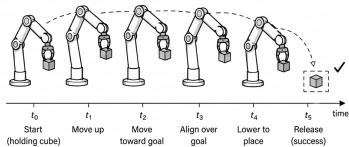
① Sample 1 → TRAJECTORY (FAILURE)

✗



② Sample 2 → TRAJECTORY (SUCCESS)

✓



Monte Carlo Value Estimation

Monte Carlo Value Estimation

Given a policy π , our objective is to estimate the value function

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s].$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo Value Estimation

Given a policy π , our objective is to estimate the value function

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s].$$

Since the true expectation is unknown, we approximate it by averaging sampled returns:

For a state s , the value estimate is

$$\hat{V}^\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s).$$

where $G_t^{(i)}$ is the return observed for the i -th visit to state s (from the state onward) and $N(s)$ is the number of visits to s

Each rollout provides one sample of the return. By the **Law of Large Numbers**, $\hat{V}_N(s) \longrightarrow V^\pi(s)$ as $N \rightarrow \infty$.

Anytime: The estimate improves as more rollouts are collected.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

First-Visit vs. Every-Visit Monte Carlo

Consider the trajectory

$$\tau = s_0, a_1, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T$$

Key Idea

Use observed reward-to-go of the state as the direct evidence of the actual expected utility of that state.

Two common approaches exist:

- **First-Visit Monte Carlo**
 - Use only the first occurrence of each state within an episode.
- **Every-Visit Monte Carlo**
 - Use every occurrence of the state when computing the average return.

Both estimators converge to the correct value, although their statistical properties differ.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example: Estimating a State Value

Suppose the following returns were observed for state s :

Episode	$G(s)$
1	8
2	6
3	10

The Monte Carlo estimate is

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example: Estimating a State Value

Suppose the following returns were observed for state s :

Episode	$G(s)$
1	8
2	6
3	10

The Monte Carlo estimate is

$$\hat{V}(s) = \frac{8 + 6 + 10}{3} = 8.$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

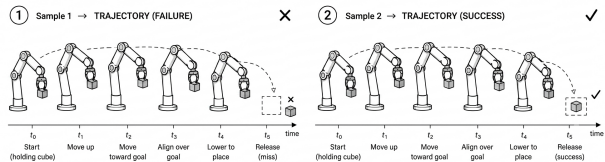
Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

First-Visit Monte Carlo Policy Evaluation

Algorithm 1 First-Visit Monte Carlo Policy Evaluation

```
1: Initialize:
2:  $\pi \leftarrow$  policy to be evaluated
3:  $V_\pi(s) \leftarrow$  arbitrary value for all  $s \in \mathcal{S}$ 
4:  $Returns(s) \leftarrow \emptyset$  for all  $s \in \mathcal{S}$ 
5: while true do
6:   Generate an episode following policy  $\pi$ 
7:   for all states  $s$  appearing in the episode do
8:      $G \leftarrow$  return following the first occurrence of  $s$ 
9:     Append  $G$  to  $Returns(s)$ 
10:     $V_\pi(s) \leftarrow \text{average}(Returns(s))$ 
11:   end for
12: end while
```

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

First-Visit Monte Carlo Policy Evaluation

Algorithm 2 First-Visit Monte Carlo Policy Evaluation for Q_π

```
1: Initialize:
2:  $\pi \leftarrow$  policy to be evaluated
3:  $Q_\pi(s, a) \leftarrow$  arbitrary value for all  $s \in \mathcal{S}, a \in \mathcal{A}$ 
4:  $Returns(s, a) \leftarrow \emptyset$  for all  $s \in \mathcal{S}, a \in \mathcal{A}$ 
5: while true do
6:   Generate an episode following policy  $\pi$ 
7:   for all state-action pairs  $(s, a)$  appearing in the episode do
8:      $G \leftarrow$  return following the first occurrence of  $(s, a)$ 
9:     Append  $G$  to  $Returns(s, a)$ 
10:     $Q_\pi(s, a) \leftarrow \text{average}(Returns(s, a))$ 
11:   end for
12: end while
```

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Challenges

Challenges with Monte Carlo Evaluation

Monte Carlo learns from complete sampled episodes.

Monte Carlo methods are simple and unbiased, but they can be slow and noisy.

- Sample inefficiency
- Diminishing returns
- Delayed rewards
- High variance

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo evaluation requires complete episodes before updating value estimates. Thus,

- Each estimate $\hat{V}_\pi(s)$ requires sampled rollouts.
- Long episodes delay learning.
- Many episodes may be needed before estimates stabilize.
- Partial trajectories are not immediately used for updates.

This motivates methods that update before the episode ends

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Diminishing Returns

Increasing the number of Monte Carlo rollouts improves the estimate, but each additional rollout contributes a smaller improvement.

The uncertainty of the estimate decreases proportionally to $\frac{1}{\sqrt{N}}$, where N is the number of sampled episodes.

Episodes	Relative Error
100	1.00
400	0.50
1600	0.25
6400	0.125

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

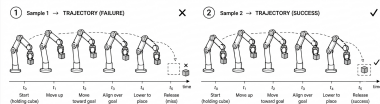
Challenges of Delayed Rewards

Suppose the only reward is received at the end of the episode:

$$r_T = 1, \quad r_t = 0 \quad \forall t < T.$$

Two related effects make learning difficult:

- Credit assignment problem
- Discounted learning signal



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Credit Assignment Problem

- A terminal reward provides only indirect feedback.
- It is difficult to determine which earlier actions caused success or failure.
- The longer the episode, the harder this attribution becomes.

Discounted Learning Signal

For a successful episode $G_t = \gamma^{T-t}$

- The reward signal decays exponentially with distance.
- Early states receive only a small learning signal.

Challenges of Delayed Rewards

Discounting weakens the learning signal

$$s_0, r_0 = 0 \rightarrow s_1, r_1 = 0 \rightarrow s_2, r_2 = 0 \rightarrow \dots \rightarrow s_{T-1}, r_{T-1} = 0 \rightarrow s_T, r_T = 1$$

The Monte Carlo return propagates backward:

$$G_{T-1} = \gamma$$

$$G_{T-2} = \gamma^2$$

$$\vdots$$

$$G_0 = \gamma^T$$

Distance from reward	1	10	50	100
$\gamma = 0.95$	0.95	0.60	0.077	0.006

The reward reaches every state on the trajectory, but its influence decreases exponentially.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte Carlo estimates are based on random rollouts.

When used in an uncertain environment, different episodes may produce different returns.

The variance of a random variable X is

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] .$$

For Monte Carlo returns,

$$\text{Var}(G_t) = \mathbb{E}[(G_t - V\pi(s_t))^2] .$$

High Variance in Monte Carlo

Monte Carlo uses the full return:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-t-1} r_T.$$

This return depends on all future randomness in the episode:

- stochastic actions,
- stochastic transitions,
- stochastic rewards,
- different episode lengths.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

An estimator is **biased** if its expected value differs from the true quantity.

For an estimator $\hat{\theta}$ of a parameter θ ,

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

Applied to value estimation,

$$\text{Bias}(\hat{V}(s)) = \mathbb{E}[\hat{V}(s)] - V\pi(s).$$

Monte Carlo estimation is **unbiased**:

$$\mathbb{E}[\hat{V}(s)] = V\pi(s)$$

But it often has **high variance**.

Monte Carlo evaluation is attractive because it is simple and unbiased.

However, it has several important challenges:

- It requires complete episodes.
- Long-horizon rewards create difficult credit assignment.
- Discounting weakens distant reward signals.
- Returns can have high variance.
- Reducing error requires many additional samples.

These limitations motivate Temporal Difference learning.

Monte Carlo Control

Monte-Carlo: Prediction (Evaluation)

Algorithm 3 First-Visit Monte Carlo Policy Evaluation

```
1: Initialize:
2:  $\pi \leftarrow$  policy to be evaluated
3:  $V_\pi(s) \leftarrow$  arbitrary value for all  $s \in \mathcal{S}$ 
4:  $Returns(s) \leftarrow \emptyset$  for all  $s \in \mathcal{S}$ 
5: while true do
6:   Generate an episode following policy  $\pi$ 
7:   for all states  $s$  appearing in the episode do
8:      $G \leftarrow$  return following the first occurrence of  $s$ 
9:     Append  $G$  to  $Returns(s)$ 
10:     $V_\pi(s) \leftarrow \text{average}(Returns(s))$ 
11:   end for
12: end while
```

How can we use this for control ?

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Algorithm 4 Monte Carlo Control with Exploring Starts

```
1: Initialize:
2:  $Q(s, a) \leftarrow$  arbitrary value for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ 
3:  $\pi(s) \leftarrow$  arbitrary action for all  $s \in \mathcal{S}$ 
4:  $Returns(s, a) \leftarrow \emptyset$  for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ 
5: while true do
6:   Choose  $S_0 \in \mathcal{S}$  and  $A_0 \in \mathcal{A}(S_0)$  such that every state-
     action pair has nonzero probability of being selected
7:   Generate an episode starting from  $(S_0, A_0)$ , following pol-
     icy  $\pi$ 
8:   for all state-action pairs  $(s, a)$  appearing in the episode do
9:      $G \leftarrow$  return following the first occurrence of  $(s, a)$ 
10:    Append  $G$  to  $Returns(s, a)$ 
11:     $Q(s, a) \leftarrow \text{average}(Returns(s, a))$ 
12:   end for
13:   for all states  $s$  appearing in the episode do
14:      $\pi(s) \leftarrow \arg \max_a Q(s, a)$ 
15:   end for
16: end while
```

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Algorithm 5 Monte Carlo Control with Exploring Starts

```
1: Initialize:
2:  $Q(s, a) \leftarrow$  arbitrary value for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ 
3:  $\pi(s) \leftarrow$  arbitrary action for all  $s \in \mathcal{S}$ 
4:  $Returns(s, a) \leftarrow \emptyset$  for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ 
5: while true do
6:   Choose  $S_0 \in \mathcal{S}$  and  $A_0 \in \mathcal{A}(S_0)$  such that every state-
     action pair has nonzero probability of being selected
7:   Generate an episode starting from  $(S_0, A_0)$ , following pol-
     icy  $\pi$ 
8:   for all state-action pairs  $(s, a)$  appearing in the episode do
9:      $G \leftarrow$  return following the first occurrence of  $(s, a)$ 
10:    Append  $G$  to  $Returns(s, a)$ 
11:     $Q(s, a) \leftarrow \text{average}(Returns(s, a))$ 
12:   end for
13:   for all states  $s$  appearing in the episode do
14:      $\pi(s) \leftarrow \arg \max_a Q(s, a)$ 
15:   end for
16: end while
```

Problem with this approach?

Monte-Carlo: From Prediction (Evaluation) to Control

- Use a random policy to simulate many (!) trajectories
- Compute the q-value of each state-action pair
- Update π by taking the max action.

Problem with this approach?

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte-Carlo Policy Control - Taxi

```
+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+-----+
(North)
num episodes completed: 1
total rewards: -133
mean rewards per episode: -133.00
```

```
def build_decision_dict(raw_data):
    # Nested dictionary for: State -> Action -> Reward List
    state_action_scores = defaultdict(lambda: defaultdict(lambda: []))
    for trajectory in raw_data:
        reward_sum = 0
        # iterate backwards to calculate the return G of each observed state action pair
        for state, action, reward in reversed(list(zip(trajectory.observations, trajectory.actions, trajectory.rewards))):
            reward_sum += reward
            state_action_scores[state][action].append(reward_sum)

    for state, action_values in state_action_scores.items():
        for action, values_list in action_values.items():
            # Calculate the mean of all returns for a state action pair
            state_action_scores[state][action] = np.mean(values_list)
        # For each state choose the action with the highest mean return
        state_action_scores[state] = max(state_action_scores[state], key=state_action_scores[state].get)
    return state_action_scores
```

https://github.com/CLAIR-LAB-TECHNION/FSTMA-course/blob/main/tutorials/tut03/Monte_Carlo.ipynb

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte-Carlo Control - Taxi

```
env.seed(seed)
```

```
policy = calc_final_policy(RandomTraveler, 100, "mcc_100")
evaluate_and_print(policy)
policy = calc_final_policy(RandomTraveler, 1000, "mcc_1000")
evaluate_and_print(policy)
policy = calc_final_policy(RandomTraveler, 10000, "mcc_10000")
evaluate_and_print(policy)
policy = calc_final_policy(RandomTraveler, 100000, "mcc_100000")
evaluate_and_print(policy)
```

```
100% ██████████ 10000/10000 [00:01<00:00, 5139.08it/s]
Monte Carlo Control Policy
-----
total reward over all episodes: -1306708
mean reward per episode:      -130.6708

100% ██████████ 10000/10000 [00:01<00:00, 5334.19it/s]
Monte Carlo Control Policy
-----
total reward over all episodes: -1242335
mean reward per episode:      -124.2335

100% ██████████ 10000/10000 [00:01<00:00, 6415.93it/s]
Monte Carlo Control Policy
-----
total reward over all episodes: -729155
mean reward per episode:      -72.9155

100% ██████████ 10000/10000 [00:01<00:00, 7840.29it/s]
Monte Carlo Control Policy
-----
total reward over all episodes: -154428
mean reward per episode:      -15.4428
```

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

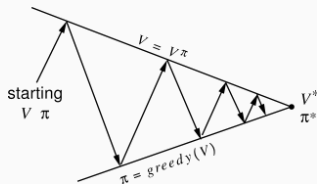
Monte Carlo
Control

Guarantees

Conclusion

Limitations?

Generalised Policy Iteration With Monte-Carlo Evaluation



Policy evaluation: Monte-Carlo policy evaluation, $V = V_\pi$?

Policy improvement: Greedy policy improvement?

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Example of Greedy Action Selection

There are two doors in front of you:

You open the left door and get reward 0

$$V(\text{left}) = 0$$

You open the right door and get reward +1

$$V(\text{right}) = +1$$

You open the right door and get reward +2

$$V(\text{right}) = +2$$

You open the right door and get reward +2

$$V(\text{right}) = +2$$

.
. .
.

Are you sure you've chosen the best door?



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

- Simplest idea for ensuring continual exploration
- All m actions are tried with non-zero probability
- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

Model Free RL:
The Setup

Monte Carlo
Value Estimation

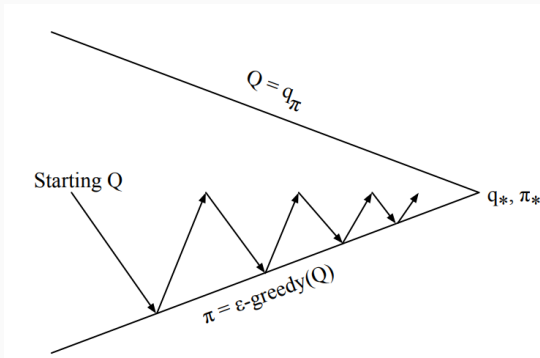
Challenges

Monte Carlo
Control

Guarantees

Conclusion

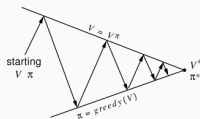
Monte-Carlo Control



Every Iteration:

- Policy evaluation: Monte-Carlo policy evaluation. $Q \approx q_\pi$
- Policy improvement: ϵ -greedy improvement

Monte-Carlo Policy Iteration



- Policy evaluation: Monte-Carlo policy evaluation. $Q_{\pi}(s, a)$
- Policy improvement: ϵ -greedy improvement

Is it necessary to wait until the end of execution of all trajectories ?

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

The mean $\mu_1, \mu_2, \dots$ of a sequence $x_1, x_2, \dots$ can be computed incrementally,

$$\begin{aligned}\mu_k &= \frac{1}{k} \sum_{j=1}^k x_j \\ &= \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right) \\ &= \frac{1}{k} (x_k + (k-1)\mu_{k-1}) \\ &= \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1})\end{aligned}$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Incremental Monte-Carlo Updates

- Update $V(s)$ incrementally after episode $S_1, A_1, R_2, \dots, S_T$
- For each state S_t with return **return** _{t} :

$$N(S_t) \leftarrow N(S_t) + 1$$

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}(\text{return}_t - V(S_t))$$

- In non-stationary problems, it can be useful to track a running mean, i.e. forget old episodes.

$$V(S_t) \leftarrow V(S_t) + \alpha(\text{return}_t - V(S_t))$$

where α is the [decaying] learning rate

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Monte-Carlo Control

Algorithm 6 On-Policy First-Visit Monte Carlo Control (ϵ -soft)

```
1: Algorithm parameter:  $\epsilon > 0$ 
2: Initialize:
3:  $\pi \leftarrow$  arbitrary  $\epsilon$ -soft policy
4:  $Q(s, a) \leftarrow$  arbitrary, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ 
5:  $Returns(s, a) \leftarrow \emptyset$ , for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ 
6: while true do
7:   Generate an episode following  $\pi$ :
       $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 
8:    $G \leftarrow 0$ 
9:   for  $t = T - 1, T - 2, \dots, 0$  do
10:     $G \leftarrow \gamma G + r_{t+1}$ 
11:    if  $(s_t, a_t)$  is the first occurrence in the episode then
12:      Append  $G$  to  $Returns(s_t, a_t)$ 
13:       $Q(s_t, a_t) \leftarrow \text{average}(Returns(s_t, a_t))$ 
14:       $a^* \leftarrow \arg \max_a Q(s_t, a)$ 
15:      for all  $a \in \mathcal{A}(s_t)$  do
16:        if  $a = a^*$  then
17:           $\pi(a|s_t) \leftarrow 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s_t)|}$ 
18:        else
19:           $\pi(a|s_t) \leftarrow \frac{\epsilon}{|\mathcal{A}(s_t)|}$ 
20:        end if
21:      end for
22:    end if
23:  end for
24: end while
```

Will this converge to an optimal policy?

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Guarantees

Policy Improvement Theorem

Definition

For policy π and nondeterministic policy $\pi' : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, define:

$$Q_{\pi}(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a|s) Q_{\pi}(s, a)$$

Theorem

Let π and π' be any pair of policies (deterministic or nondeterministic). If for all $s \in \mathcal{S}$ $Q_{\pi}(s, \pi'(s)) \geq V_{\pi}(s)$ then for all $s \in \mathcal{S}$, $V_{\pi'}(s) \geq V_{\pi}(s)$. i.e., π' is an improvement over π .

π' , that can either exploit or explore, must be at least as good as π

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

ϵ -Greedy Policy Improvement

Definition

A nondeterministic policy π is ϵ -**soft** if

$$\forall s \in \mathcal{S}, a \in \mathcal{A} \quad \pi(a|s) \geq \frac{\epsilon}{|\mathcal{A}|}$$

Definition

A nondeterministic policy π is called ϵ -**greedy** with respect to Q if for some action $a^* \in \arg \max_{a \in \mathcal{A}} Q(s, a)$

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & a = a^* \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases}$$

Theorem

For any ϵ -soft policy π , if the ϵ -greedy policy π' with respect to Q_π , then $V_{\pi'}(s) \geq V_\pi(s)$.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

ϵ -Greedy Policy Improvement

Theorem

For any ϵ -soft policy π , if the ϵ -greedy policy π' with respect to Q_π , then $V_{\pi'}(s) \geq V_\pi(s)$.

$$Q_\pi(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a|s) Q_\pi(s, a) \quad \text{definition}$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

ϵ -Greedy Policy Improvement

Theorem

For any ϵ -soft policy π , if the ϵ -greedy policy π' with respect to Q_π , then $V_{\pi'}(s) \geq V_\pi(s)$.

$$Q_\pi(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a|s) Q_\pi(s, a) \quad \text{definition}$$

$$= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} Q_\pi(s, a) \quad \text{definition}$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

ϵ -Greedy Policy Improvement

Theorem

For any ϵ -soft policy π , if the ϵ -greedy policy π' with respect to Q_π , then $V_{\pi'}(s) \geq V_\pi(s)$.

$$Q_\pi(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a|s) Q_\pi(s, a) \quad \text{definition}$$

$$= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} Q_\pi(s, a) \quad \text{definition}$$

Since π is ϵ -soft, then $\frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}|}}{1 - \epsilon}$ is non-negative and sums to 1.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

ϵ -Greedy Policy Improvement

Theorem

For any ϵ -soft policy π , if the ϵ -greedy policy π' with respect to Q_π , then $V_{\pi'}(s) \geq V_\pi(s)$.

$$Q_\pi(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a|s) Q_\pi(s, a) \quad \text{definition}$$

$$= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} Q_\pi(s, a) \quad \text{definition}$$

Since π is ϵ -soft, then $\frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}|}}{1 - \epsilon}$ is non-negative and sums to 1.

Since $\max Q_\pi$ is greater than any weighted sum of Q_π , then

$$\geq \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q_\pi(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \frac{\epsilon}{m}}{1 - \epsilon} Q_\pi(s, a)$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

ϵ -Greedy Policy Improvement

Theorem

For any ϵ -soft policy π , if the ϵ -greedy policy π' with respect to Q_π , then $V_{\pi'}(s) \geq V_\pi(s)$.

$$Q_\pi(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a|s) Q_\pi(s, a) \quad \text{definition}$$

$$= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} Q_\pi(s, a) \quad \text{definition}$$

Since π is ϵ -soft, then $\frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}|}}{1 - \epsilon}$ is non-negative and sums to 1.

Since $\max Q_\pi$ is greater than any weighted sum of Q_π , then

$$\begin{aligned} &\geq \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} Q_\pi(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \frac{\epsilon}{m}}{1 - \epsilon} Q_\pi(s, a) \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) Q_\pi(s, a) = V_\pi(s) \quad \text{policy improvement} \implies V_{\pi'}(s) \geq V_\pi(s) \end{aligned}$$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

How do we make sure exploration eventually stops?

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

How do we make sure exploration eventually stops?

- Decay ϵ Over Time: e.g., exponential decay $\epsilon_t = \gamma \cdot \epsilon_{t-1}$ with $\gamma \in (0, 1)$.
- After a sufficient number of iterations (when $Q(s, a)$ has stabilized), switch to a purely greedy policy.

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

Greedy in the Limit with Infinite Exploration (GLIE)

- All state-action pairs are explored infinitely many times,

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- The policy converges on a greedy policy,

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbf{1}(a = \arg \max_{a' \in \mathcal{A}} Q_k(s, a'))$$

For example, ϵ -greedy is GLIE if α reduces to zero at $\epsilon_k = \frac{1}{k}$.

- Sample k th episode using $\pi : \{s_1, A_1, R_2, \dots, s_T\} \sim \pi$
- For each state s_t and action A_t in the episode,

$$N(s_t, A_t) \leftarrow N(s_t, A_t) + 1$$

$$Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \frac{1}{N(s_t, A_t)}(\text{return}_t - Q(s_t, A_t))$$

- Improve policy based on new action-value function

$$\epsilon \leftarrow \frac{1}{k}$$

$$\pi \leftarrow \epsilon\text{-greedy}(Q)$$

Theorem

GLIE Monte-Carlo control converges to the optimal action-value function, $Q(s, a) \rightarrow Q^*(s, a)$

Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

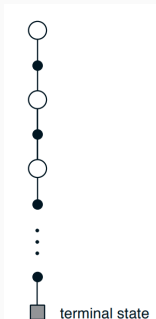
Guarantees

Conclusion

Conclusion

Monte Carlo (MC)

- MC methods learn directly from episodes of experience
- MC is model-free: no knowledge of MDP transitions / rewards
- MC learns from complete episodes: no bootstrapping.
- MC uses the simplest possible idea: value = mean return
- Caveat: can only apply MC to episodic MDPs: all episodes must terminate



Model Free RL:
The Setup

Monte Carlo
Value Estimation

Challenges

Monte Carlo
Control

Guarantees

Conclusion

- Converge very slowly to correct utilities values (requires a lot of sequences)

$$V_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

- Doesn't exploit Bellman constraints on policy values

$$V_{\pi}(s) = R(s) + \gamma \sum_{s'} \mathcal{P}(s'|s, \pi(s)) \cdot V_{\pi}(s')$$

- Can consider estimates that violate this property badly

How can we incorporate such constraints ?